HOME HUB · MAIN DESIGN DOCUMENT

One dashboard for every device in the house

A self-hosted home platform with a Java backend and a modern browser UI. It brings the gate, heating, fire alarms, dehumidifier, robot vacuum, printers, cameras, network, fridge and coffee machine onto one clean dashboard. Device health and notifications are first-class features.

Status	Date	Owner	Runs on
Phase 1: Design (draft v0.1)	8 November 2026	Serkan Yucel	Docker Compose, single home server

---

1. Vision & scope
2. Principles & technology
3. System context
4. Component architecture
5. Integrating proprietary devices
6. Device integration summary
7. Health & notifications
8. Key runtime flows
9. Deployment & security
10. Roadmap: 20 phases
11. Decisions & open questions
12. Detailed design documents

1 Vision & scope

Home Hub is **not** a Home Assistant clone. Home Assistant has a decade of community integrations, and matching it is not the goal. Instead, Home Hub does a smaller set of things very well, for the devices this household actually owns. Its architecture still lets it grow through open standards.

🏠 One dashboard
Every device on one live, glanceable screen, organised by room.

🩺 Health first
You always see what is offline or faulty, why, and for how long, with an incident history.

🔔 One inbox
Notifications from every device: de-duplicated, with acknowledge/snooze, delivered by Web Push and email.

🎛️ Direct control
Open the gate, set the heating, clean rooms, print a PDF, all from the browser.

🔌 Driver SDK
New integrations in days: Java plug-ins or containers in any language.

🔒 Local & secure
Local protocols first, VPN-only remote access, encrypted credentials.

Out of scope for v1.0

- Feature parity with Home Assistant, native mobile apps (a PWA is used instead), multi-home SaaS, and NVR video recording.
- Being the *primary* fire alarm. The detectors and the official app stay primary, and the hub is a second channel.

2 Principles & technology

PRINCIPLE	WHAT IT MEANS
One internal language	Every vendor protocol is translated at the edge into `Device → Capability → State / Command / Event`. The core, UI and rules never see a vendor protocol.
Drivers are plug-ins	A driver runs in-process (Java SPI) or out-of-process (a sidecar container speaking MQTT). This lets us reuse mature community libraries.
Health is computed	Derived from heartbeats, last-seen times, fault codes and connectivity, rather than trusting vendors to report problems.
Budgets for cloud APIs	Rate limits (e.g. ViCare: 1,450 calls/day) are enforced centrally, with a reserve kept for user commands.
Secure by default	Vault-encrypted secrets, step-up confirmation for opening the gate, no port forwarding.
Boring core, exciting edges	Proven technology in the core; the innovation goes into the drivers and the user experience.

LAYER	CHOICE
Backend	Java 25 (virtual threads), Spring Boot 4, Maven multi-module (a modular monolith)
Data	PostgreSQL 17 + TimescaleDB (registry, state, time-series history), Flyway
Messaging	Typed in-process event bus; Eclipse Mosquitto (MQTT 5) at the edge for sidecars and MQTT devices
API	REST (OpenAPI 3) + WebSocket/STOMP for live updates
Frontend	React 19 + TypeScript + Vite, Tailwind + shadcn/ui, TanStack Query, ECharts; installable PWA
Media & printing	go2rtc (RTSP → WebRTC), IPP Everywhere (+ optional CUPS)
Delivery	Docker images (Jib), Docker Compose, Caddy (TLS), GitHub Actions

3 System context

Who and what Home Hub talks to. Local devices are reached directly on the LAN; cloud-only devices go through vendor APIs. Remote access goes through the existing VPN router.

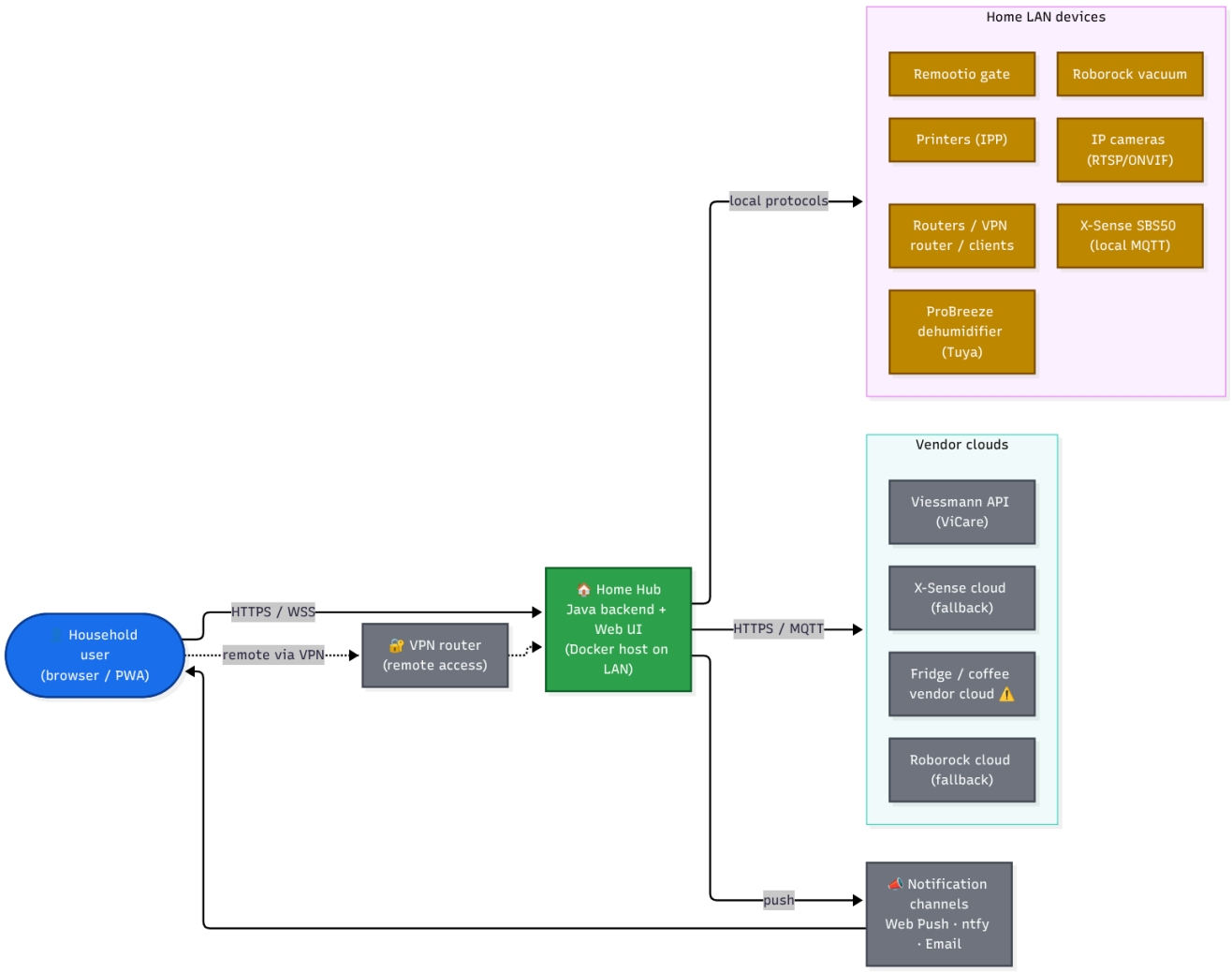


Figure 1: System context

4 Component architecture

Five layers inside one Spring Boot application, plus infrastructure containers. Commands flow down to the drivers; state and events flow up through the event bus to the subscribers and out to the browser.

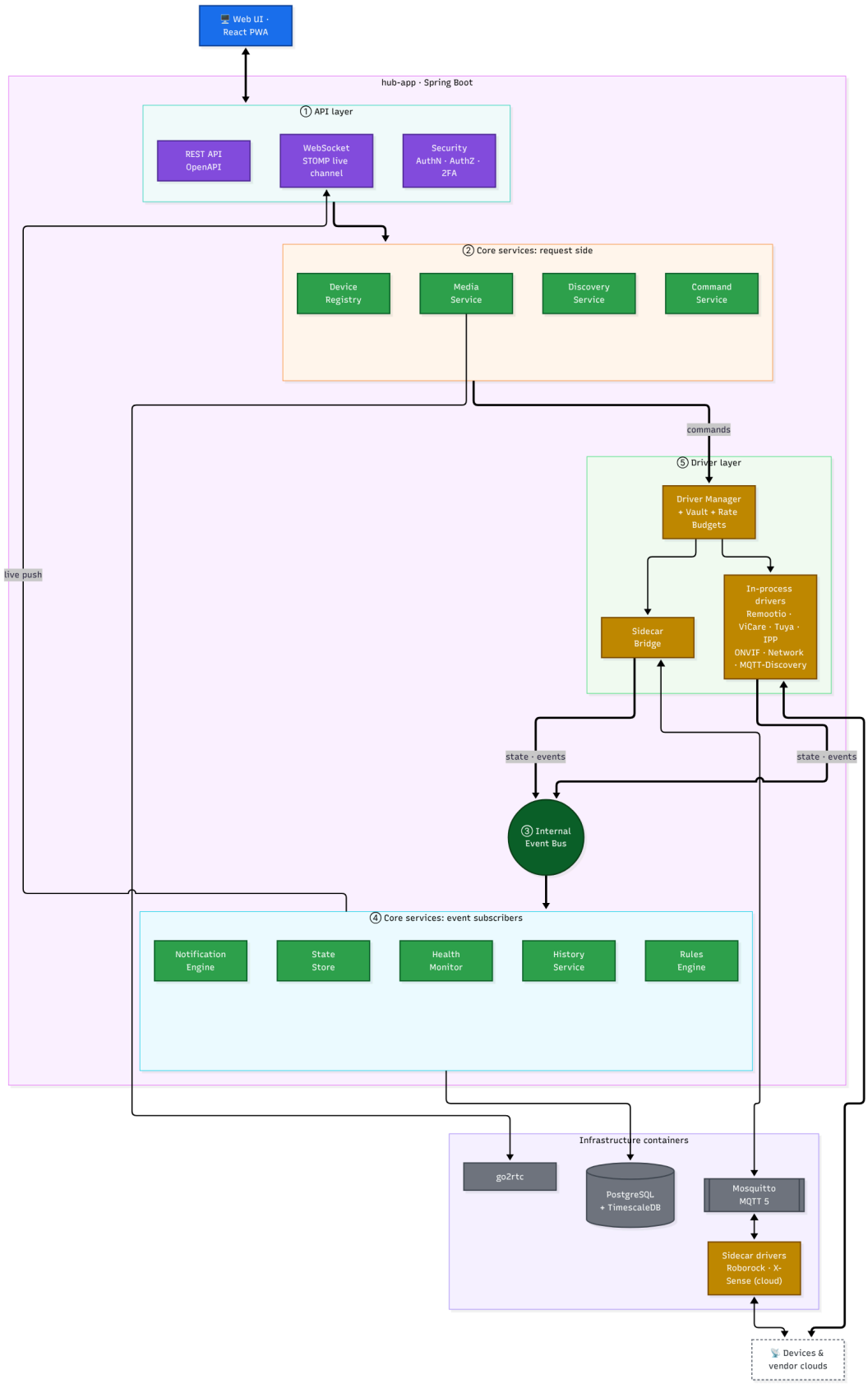


Figure 2: Component architecture (layered view)

COMPONENT	RESPONSIBILITY
API layer	REST for queries and commands; WebSocket for live state, events and notifications; authentication, roles, 2FA and step-up
Device Registry · Command Service	Devices, rooms and capabilities; validates, confirms, dispatches and tracks commands (`PENDING → SENT → CONFIRMED / FAILED / TIMEOUT`)
Event Bus	Typed in-process pub/sub with per-device ordering and isolated subscribers on virtual threads
State · Health · Notifications · History · Rules	Current-state projection; health and incidents; notification inbox and channels; TimescaleDB history; simple automations
Driver layer	Driver Manager (lifecycle, supervision, back-off restarts), Credential Vault, Rate-Budget Manager, in-process drivers, Sidecar Bridge
Media & Discovery	Camera sessions through go2rtc, print jobs; mDNS / SSDP / ONVIF / ARP discovery for onboarding

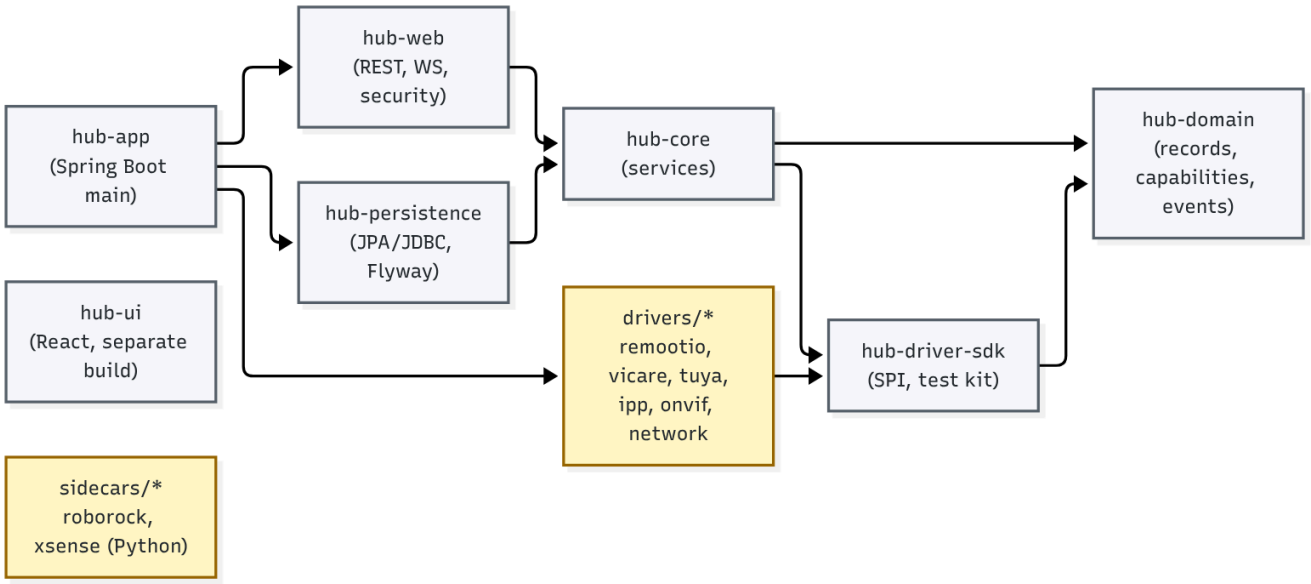


Figure 3: Maven modules. Drivers depend only on the SDK.

5 Integrating proprietary devices

Every vendor speaks its own language: encrypted WebSockets (Remootio), OAuth REST with strict quotas (Viessmann), AES-encrypted TCP (Tuya), RTSP (cameras), IPP (printers), and reverse-engineered cloud protocols (Roborock, X-Sense). Home Hub supports **two kinds of driver behind one contract**, so the core cannot tell them apart.

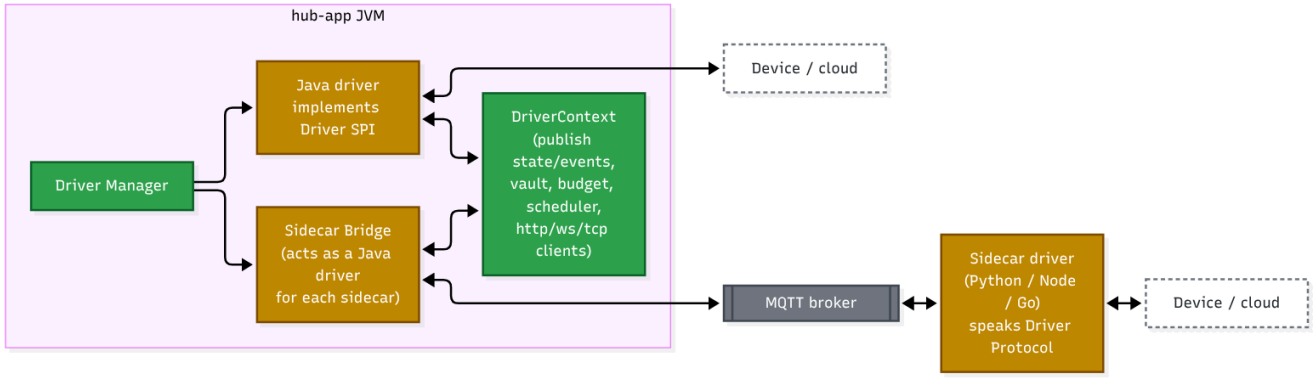


Figure 4: In-process Java drivers and out-of-process sidecars share one contract

INTEGRATION STYLE	PATTERN	USED BY
Local push	Persistent socket; listen loop; reconnect with back-off; availability on disconnect	Remootio, Tuya, ONVIF events
Local poll	Scheduled query → diff → publish changed attributes	Printers, routers, ARP
Cloud poll with quota	Rate budget checked first; adaptive interval; "cloud unreachable" handling	Viessmann ViCare
Device publishes to our broker	Home Assistant-style MQTT discovery topics → capabilities	X-Sense SBS50, Zigbee2MQTT, Tasmota, Shelly
Sidecar (any language)	MQTT Driver Protocol: state, events, commands with request/response, Last Will for liveness	Roborock, X-Sense cloud fallback
Media / jobs	The driver exposes streams or jobs; Media Service and go2rtc deliver WebRTC; IPP job tracking	Cameras, printers

6 Device integration summary

local, documented · local reverse-engineered, or official cloud · ⚠️ needs your input

DEVICE	HOW	DRIVER	PHASE
Remootio main gate local	Local WebSocket API v2 (encrypted), mDNS discovery, PIN step-up to open	Java `remootio`	12
HP & other printers local	IPP Everywhere: status, ink, print a PDF straight from the dashboard	Java `ipp`	13
Routers, VPN router, clients ⚠️	Generic ARP / mDNS / SNMP layer plus a vendor API adapter; unknown-device alerts; Wi-Fi survey	Java `network`	14
X-Sense fire alarms local	**Official local MQTT bridge** from the SBS50 base station to our broker; cloud API as a fallback for battery and liveness	Java `mqtt-discovery` + sidecar	15
Viessmann ViCare heating cloud	Official REST API, OAuth PKCE; free tier allows 120 calls/10 min and 1,450/day, so polling is adaptive	Java `vicare`	16
ProBreeze dehumidifier ⚠️	Tuya local protocol (TCP 6668, AES) with a device-profile file; Midea variant if the model needs it	Java `tuya-local`	17
Roborock vacuum local	Local TCP 58867 using a local key fetched once from the cloud; built on python-roborock	Sidecar `roborock`	18
IP cameras ⚠️	ONVIF discovery, PTZ and events; RTSP → go2rtc → WebRTC in the browser	Java `onvif` + go2rtc	19
Fridge & coffee machine ⚠️	Depends on brand: Home Connect, SmartThings or LG ThinQ (all official APIs)	Java, per vendor	20

> **Leverage:** the generic MQTT-Discovery and Tuya-local drivers also cover Zigbee2MQTT, Tasmota, Shelly and hundreds of Tuya devices *without new code*. That is the route for lights and sensors after v1.

7 Health & notifications

Health is the headline feature. Every device has a computed health state, and each transition into a bad state opens an **incident**, which feeds the "malfunction history" and uptime figures.

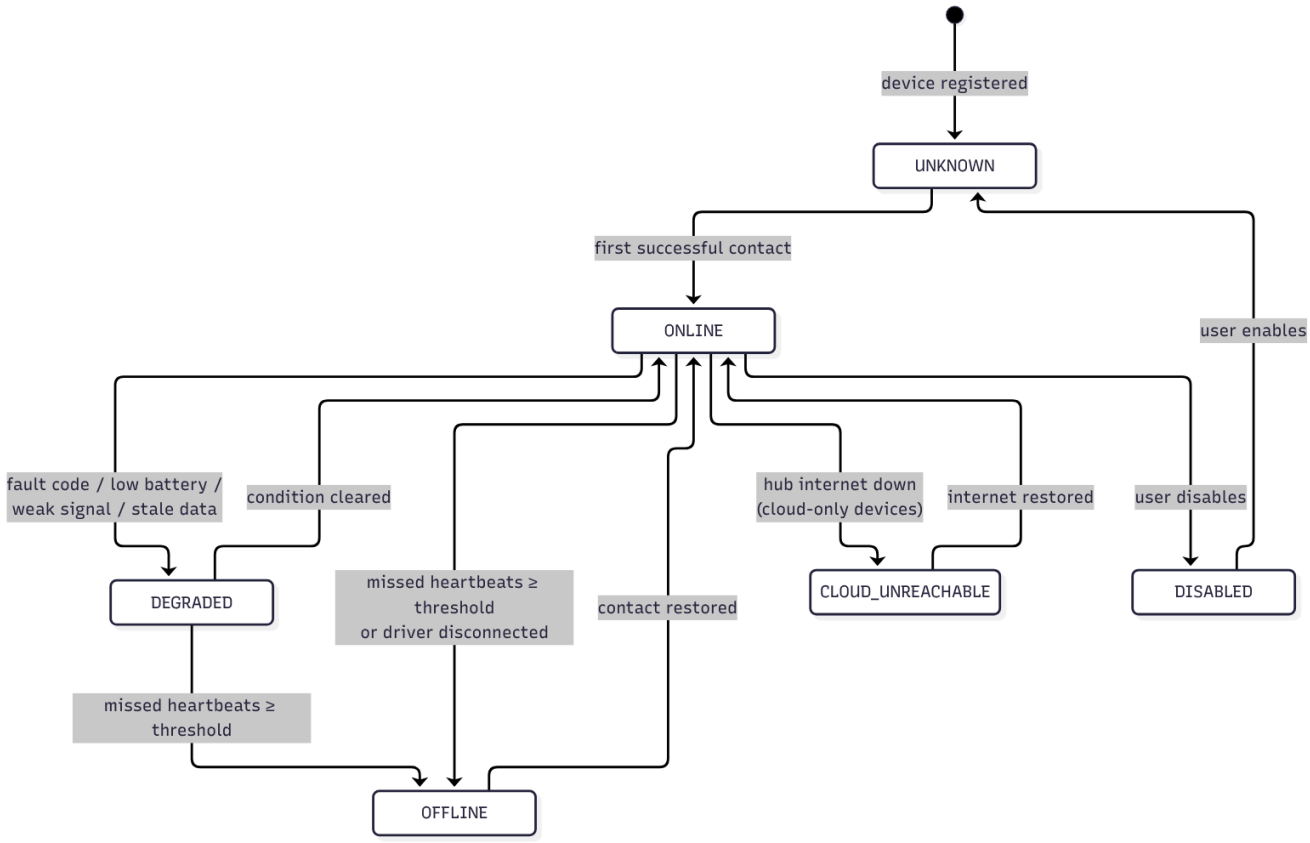


Figure 5: Device health state machine

- **OFFLINE** is declared when no data arrives for 2× the expected refresh interval, or when the driver's connection drops.
- **DEGRADED** covers fault codes, low battery, weak Wi-Fi, a full tank, or consumables under 10 %, always with a human-readable reason.
- **CLOUD_UNREACHABLE** applies when the hub's own internet is down. Cloud-only devices are not falsely reported as broken.
- **Network correlation:** "Offline: not connected to Wi-Fi (last seen on AP Upstairs, −84 dBm)".

NOTIFICATION FEATURE	BEHAVIOUR
Severity	INFO · WARNING · CRITICAL. CRITICAL (smoke, CO) bypasses quiet hours and goes to every channel and every user.
De-duplication	Repeats increment a counter instead of sending another push. A grace period prevents alerts from devices that flap on and off.
Lifecycle	New → Seen → Acknowledged → Resolved, or Snoozed. Incident-linked notifications auto-resolve.
Channels	In-app inbox (always), Web Push (VAPID), email, optional self-hosted ntfy.

8 Key runtime flows

Four of the twelve sequence diagrams. The full set is in `design/04-sequence-diagrams.md`.

8.1 Opening the main gate (dangerous command with step-up)

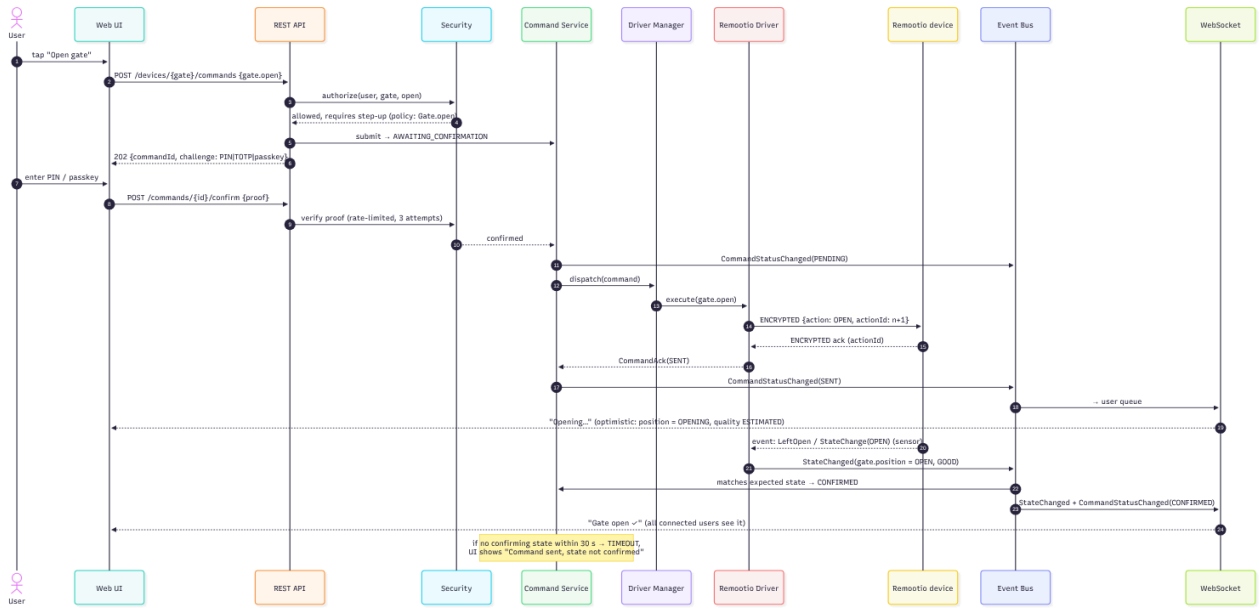


Figure 6: Open gate: PIN confirmation, encrypted local command, optimistic then confirmed state

8.2 Device event to notification (dehumidifier tank full)

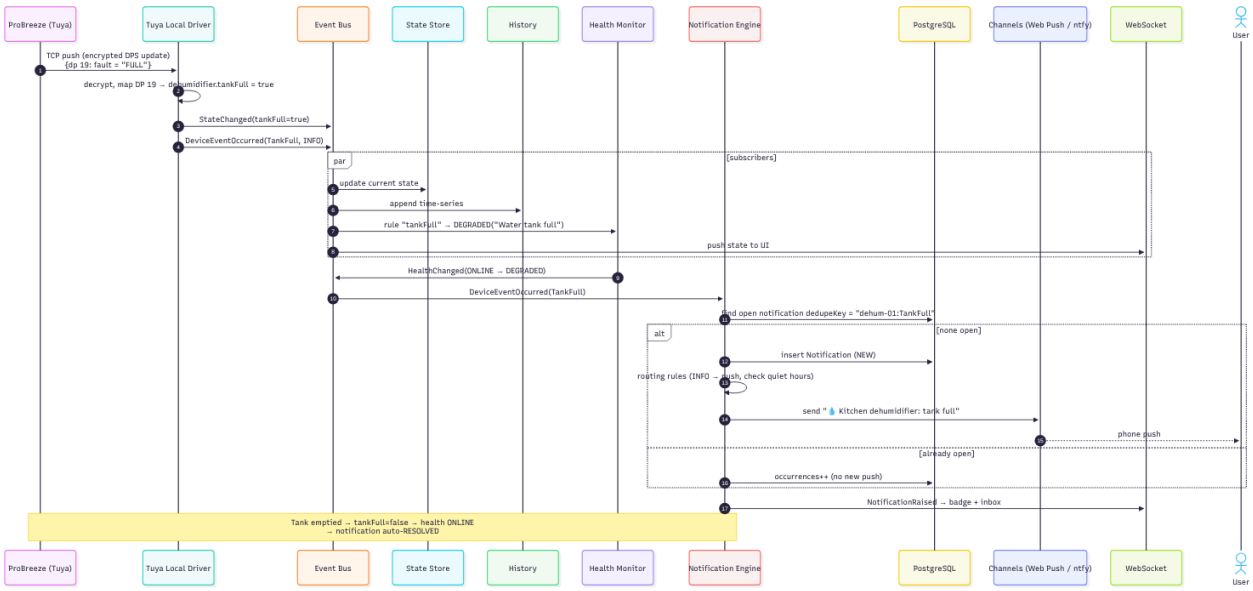


Figure 7: Push event → state, health, history, de-duplicated notification, live UI

8.3 Device goes offline and recovers

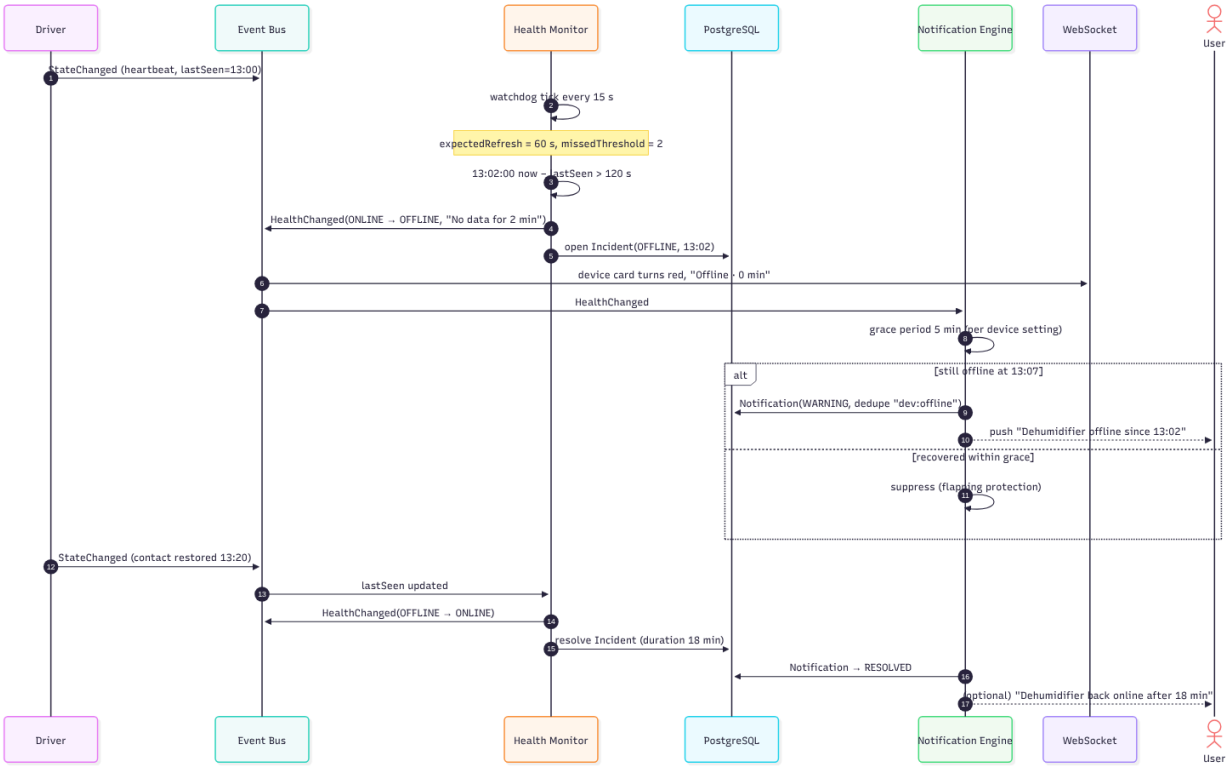


Figure 8: Heartbeat watchdog → incident → grace period → notification → auto-resolve

8.4 Fire alarm (X-Sense)

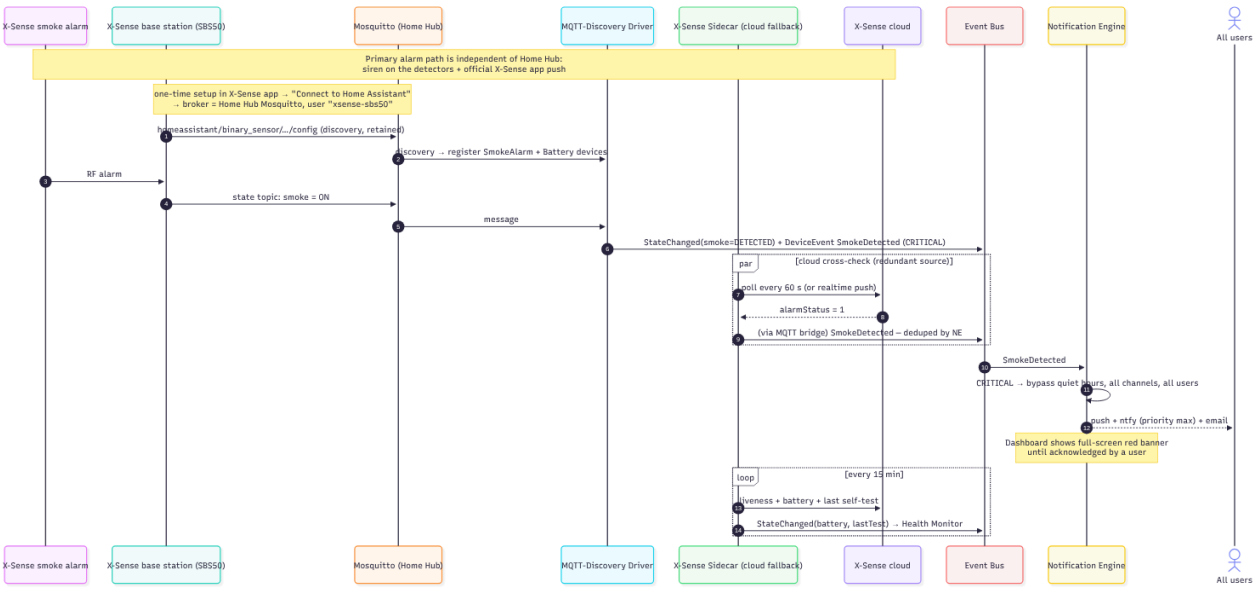


Figure 9: Local MQTT bridge with cloud cross-check and CRITICAL routing. The detectors stay primary.

9 Deployment & security

One `docker compose up` on a home server. `hub-app` uses host networking so that multicast discovery (mDNS, SSDP, ONVIF, Tuya) works; databases and brokers bind to localhost only.

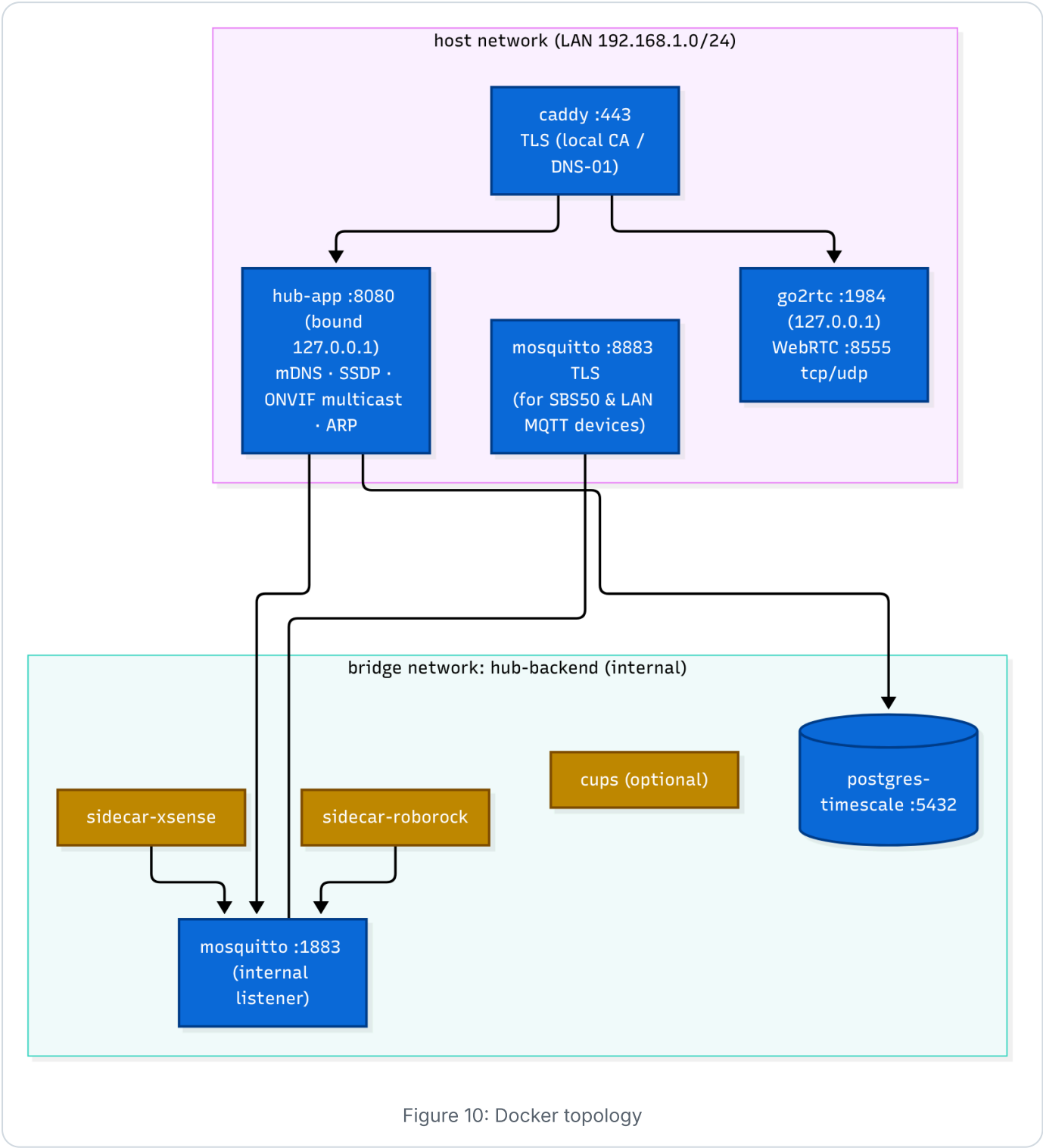


Figure 10: Docker topology

THREAT	MITIGATION
Internet access to the dashboard	No port forwarding; remote access via the VPN router only; Argon2id passwords + TOTP / passkeys
Stolen session or phone	Step-up PIN/passkey for opening the gate and other dangerous commands; session revocation; audit log
Database or backup leak	AES-256-GCM credential vault; master key kept outside the DB as a Docker secret; secrets are write-only through the API
Compromised sidecar or library	Non-root, read-only containers; per-sidecar MQTT user with topic ACLs
Cloud API lock-out	Central rate budgets with a reserve for user commands
False sense of fire safety	The hub is a secondary channel only, by design (ADR-007)

Backups: a nightly `pg_dump` → restic (encrypted), config in Git, and the master key kept in a password manager. A restore drill is part of the v1.0 exit criteria.

10 Roadmap: 20 phases

Each phase is small, testable and ends with a running system. Roughly 9–10 months part-time (S ≈ 1 week, M ≈ 2 weeks, L ≈ 3 weeks).

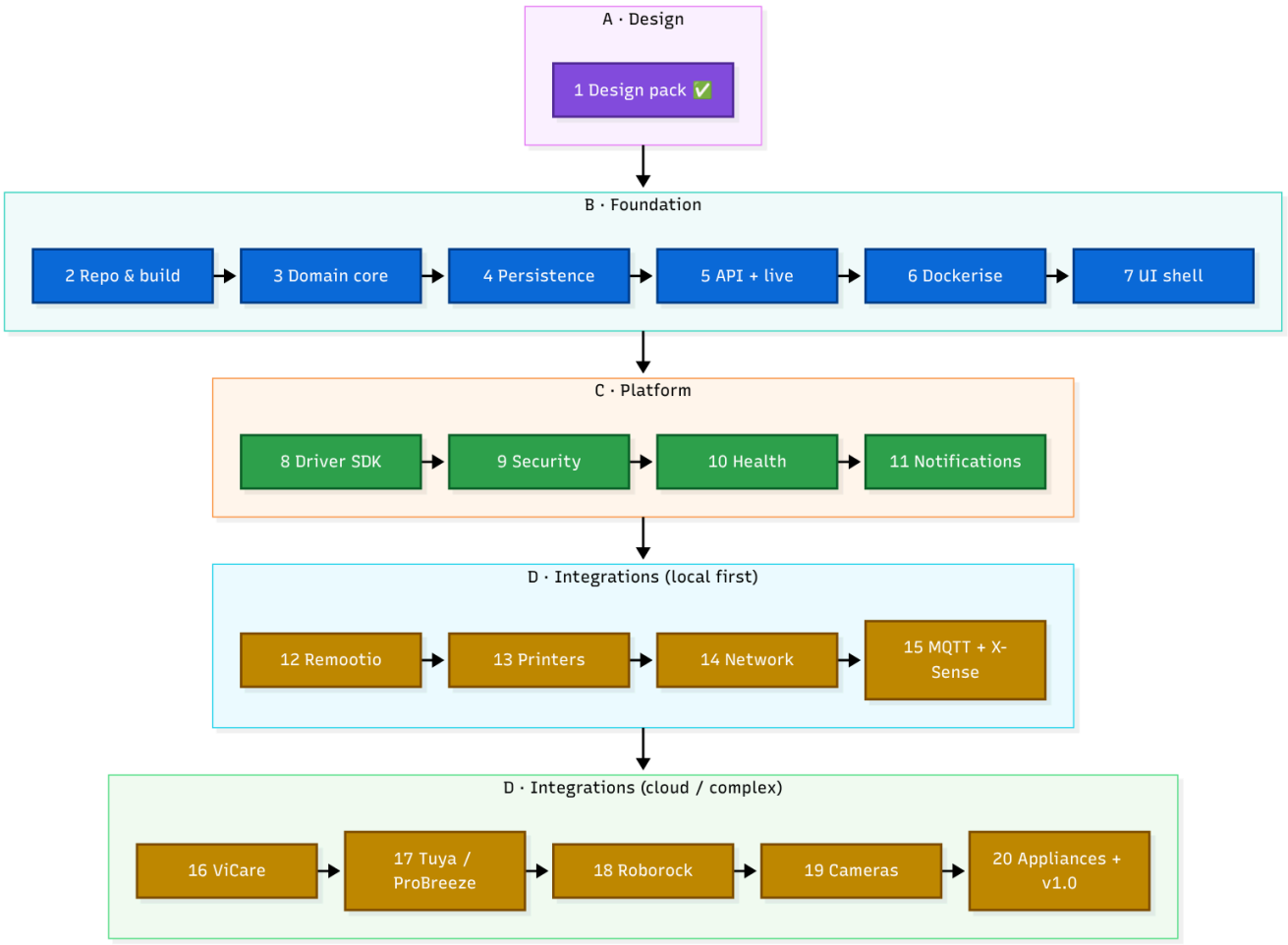


Figure 11: Phase overview

#	PHASE	DONE WHEN	SIZE
1	Design pack ✅	Pack reviewed; open questions answered or defaults accepted	S
2	Repository & build	Multi-module Maven, CI green, empty app healthy	S
3	Domain core & event bus	Capabilities, command lifecycle, ordered bus; simulated devices running	M
4	Persistence	Postgres/TimescaleDB schema; state survives restarts; history downsampled	S
5	REST + WebSocket API	Commands observable live over WebSocket; contract tests pass	M
6	Dockerise	`docker compose up` → working HTTPS hub; backup/restore tested	S
7	UI shell & design system	Live dashboard on desktop and mobile; PWA and accessibility scores ≥ 90	M
8	Driver SDK & manager	Integrations added from the UI with no UI code; crash isolation works	M
9	Security	2FA, step-up, vault, audit; ZAP baseline scan clean	M
10	Health monitor	OFFLINE detected within 2× refresh; incidents and timeline shown	M
11	Notifications	One push per issue, de-duplicated, auto-resolved	M
12	Remootio gate	Gate opens with a PIN in under 1 s; live state	S
13	Printers (IPP)	Drag-and-drop PDF prints duplex; paper-out alert	M
14	Network & routers	All clients listed; unknown-device alert; "not on Wi-Fi" diagnosis	L
15	MQTT, sidecars & X-Sense	Detector test button → CRITICAL push on all phones	M
16	ViCare heating	Target set from the dashboard; no rate-limit bans in a 7-day soak	M
17	Tuya / ProBreeze	Works with the internet unplugged; tank-full alert	M
18	Roborock (sidecar)	Room cleaning from the dashboard; error alert within 1 min	M

19	Cameras	Live view in under 2 s on the LAN and over VPN; motion thumbnails	M
20	Appliances, hardening, v1.0	Fridge and coffee machine added; 14-day soak and chaos tests pass; v1.0 released	L

After v1.0: Zigbee lights and sensors (via Zigbee2MQTT), Matter, a visual automation builder and scenes, presence, an energy dashboard, Frigate camera recording, local voice, and an optional Home Assistant bridge for long-tail devices.

11 Decisions & open questions

ADR	DECISION	STATUS
001	Java 25 + Spring Boot 4 modular monolith	Accepted
002	Capability-based device model as the single internal language	Accepted
003	In-process event bus; MQTT only at the edge	Accepted
004	Polyglot drivers: Java SPI and MQTT sidecars	Accepted
005	Docker Compose with host networking for discovery	Proposed
006	Remote access only through VPN	Proposed
007	The hub is never the primary life-safety alarm	Accepted
008	PostgreSQL + TimescaleDB only (no Redis, no separate TSDB)	Accepted
009	React + TypeScript PWA	Proposed
010	Generic MQTT-Discovery driver (Home Assistant convention)	Accepted
011	Scope: own devices first, open standards second, no HA parity	Accepted

Needs your input before the related phase: main router and VPN router models (Q1–Q2) · camera brands (Q3) · fridge and coffee machine brands (Q4–Q5) · Roborock model (Q6) · ProBreeze model and its app (Q7) · Remootio version and gate sensor (Q8) · boiler model and API tier (Q9) · X-Sense detector models (Q10) · printer models (Q11) · Docker host hardware (Q12) · household users (Q13) · own domain for TLS (Q14). Each question has a safe default, listed in `design/08-decisions-and-open-questions.md`.

12 Detailed design documents

This document is the summary. Full detail lives in the `design/` folder:

DOCUMENT	CONTENTS
`design/00-README.md`	Index and reading guide
`design/01-architecture-overview.md`	Goals, principles, stack, context, component, module and deployment views, quality targets
`design/02-domain-model.md`	Class diagram, capability catalogue, health and command state machines, notification model, ER schema
`design/03-driver-sdk-and-messaging.md`	Java SPI, MQTT Driver Protocol, event bus guarantees, rate-budget manager
`design/04-sequence-diagrams.md`	12 sequence diagrams covering every major flow
`design/05-integration-specs.md`	Per-device specs: transport, auth, capabilities, health signals, risks
`design/06-security-and-deployment.md`	Threat model, auth, vault, Docker topology and Compose file, TLS, backups, operations
`design/07-ui-ux-design.md`	Information architecture, dashboard wireframe, device card states, design system
`design/08-decisions-and-open-questions.md`	Full ADRs and the open-question list with defaults
`design/09-roadmap.md`	Full 20-phase plan with scope, exit criteria and timeline